

ORBIS SCHEDULER

API Endpoint Reference

Multi-Tenant Service Booking & Scheduling Platform

Author	Hussain Jameel
Subject	ITA602 — Advanced IT Work Integrated Learning
Institution	Torrens University Australia
API Style	RESTful HTTP / JSON
Total Endpoints	30 routes across 7 groups
Document Status	Updated to match shipped codebase — supersedes the 27-route and 29-route drafts
Last Updated	18 August 2026

Table of Contents

1. Overview.....	3
2. All Endpoints — Quick Reference.....	4
Group 1 — Auth.....	4
Group 2 — Business Profile.....	4
Group 3 — Availability.....	4
Group 4 — Booking Forms & Fields.....	4
Group 5 — Slots.....	5
Group 6 — Bookings.....	5
Group 7 — Admin.....	5
Endpoint Details — Group 1 — Auth.....	6
Endpoint Details — Group 2 — Business Profile.....	9
Endpoint Details — Group 3 — Availability.....	11
Endpoint Details — Group 4 — Booking Forms & Fields.....	13
Endpoint Details — Group 5 — Slots.....	16
Endpoint Details — Group 6 — Bookings.....	17
Endpoint Details — Group 7 — Admin.....	21
3. HTTP Status Code Reference.....	25
4. Documentation Corrections.....	25
5. Key Design Decisions.....	26

1. Overview

Orbis Scheduler exposes a RESTful JSON API organised into four access tiers, each protected by a distinct authentication strategy.

Prefix	Auth	Used by
/api/auth	None	Owner registration, login, password reset
/api/public	None	Customers — booking page, slots, booking submission
/api/owner	JWT (role = owner)	Business owners — profile, availability, forms, bookings
/api/admin	JWT (role = admin)	Platform admin — registrations, platform stats, business management, own identity

Revision note: The endpoint count has drifted across earlier drafts of this document — first 27, then 29. The real, shipped total is 30. The 30th route, `GET /api/admin/me`, was added after the original 7 admin routes were built, once the admin sidebar needed to display the logged-in admin's own name and email (mirroring `GET /api/owner/business` for the owner sidebar). This revision reconciles the document with the live codebase.

2. All Endpoints — Quick Reference

Group 1 — Auth

Method	Endpoint	Description
POST	/api/auth/register	Business owner signs up
POST	/api/auth/login	Owner or admin logs in
POST	/api/auth/forgot-password	Request password reset link
POST	/api/auth/reset-password	Submit new password with token

Group 2 — Business Profile

Method	Endpoint	Description
GET	/api/owner/business	Get own business profile
PATCH	/api/owner/business	Update own business profile
GET	/api/public/businesses/:businessId	Load public booking page data

Group 3 — Availability

Method	Endpoint	Description
GET	/api/owner/availability	Get weekly availability schedule
PUT	/api/owner/availability	Save weekly availability schedule

Group 4 — Booking Forms & Fields

Method	Endpoint	Description
GET	/api/owner/form	Get active form and all fields
PUT	/api/owner/form	Update form title, description, window
POST	/api/owner/form/fields	Add a new field to the form
PATCH	/api/owner/form/fields/:id	Edit an existing field
DELETE	/api/owner/form/fields/:id	Delete a field
PUT	/api/owner/form/fields/reorder	Reorder fields

Group 5 — Slots

Method	Endpoint	Description
GET	/api/public/slots?businessId=&date=	Get available slots for a date

Group 6 — Bookings

Method	Endpoint	Description
POST	/api/public/bookings	Customer submits a booking
GET	/api/owner/bookings	List all bookings with filters and status counts
GET	/api/owner/bookings/:id	Get single booking full details
PATCH	/api/owner/bookings/:id/approve	Approve a booking
PATCH	/api/owner/bookings/:id/reject	Reject a booking
PATCH	/api/owner/bookings/:id/cancel	Cancel an approved booking

Group 7 — Admin

Method	Endpoint	Description
GET	/api/admin/me	Get logged-in admin's own name and email (new since v1)
GET	/api/admin/stats	Platform overview numbers
GET	/api/admin/businesses	List all businesses
GET	/api/admin/businesses/:id	Single business full details
PATCH	/api/admin/businesses/:id/approve	Approve a registration
PATCH	/api/admin/businesses/:id/reject	Reject a registration
PATCH	/api/admin/businesses/:id/suspend	Suspend a business
PATCH	/api/admin/businesses/:id/activate	Activate a business

Endpoints highlighted above and marked "(new since v1)" were added after the original design docs were written.

Group 1 — Auth

POST /api/auth/register

Auth required	None — public route
Actor	Business Owner (new)
Use case	UC4 — Business Owner Registers and Sets Up Account

Request Body

```
{
  "ownerName": "Joe Smith",           // required
  "email": "joe@example.com",         // required – login email
  "password": "secret123",            // required – min 8 chars
  "businessName": "Joes Plumbing",    // required
  "businessPhone": "0412345678",      // optional
  "businessEmail": "joe@joesplumbing.com.au", // optional
  "description": "Sydney plumber",    // optional
  "websiteUrl": "https://joesplumbing.com.au" // optional
}
```

Response — 201 Created

```
{
  "message": "Registration submitted. Awaiting admin approval."
}
```

Note: No JWT is issued at registration. The owner cannot log in until an admin approves the business. A notification email is sent to the admin and a confirmation email to the owner.

POST /api/auth/login

Auth required	None — public route
Actor	Business Owner or Platform Admin
Use case	UC4 (owner login), UC10 (admin login)

Request Body

```
{
  "email": "joe@example.com",
  "password": "secret123"
}
```

200 OK — credentials valid, business approved and active:

```
{
  "token": "eyJ...",
  "role": "owner"
}
```

403 — credentials valid but registration still pending:

```
{
  "message": "Your registration is still under review. We will email you once approved."
}
```

403 — credentials valid but registration was rejected:

```
{
  "message": "Your registration was not approved.",
  "reason": "Business type not supported on this platform."
}
```

403 — credentials valid but account suspended:

```
{
  "message": "Your account has been suspended. Please contact support."
}
```

401 — wrong credentials:

```
{
  "message": "Invalid credentials."
}
```

Note: The JWT payload contains `userId`, `businessId` (owners only) and `role`. Admin and owner share the same login route — `role` in the token determines access.

POST /api/auth/forgot-password

Auth required	None — public route
Actor	Business Owner
Use case	UC15 — Forgot Password

Request Body

```
{
  "email": "joe@example.com"
}
```

Response — 200 OK

```
{
  "message": "If this email is registered you will receive a reset link shortly."
}
```

Note: The response is intentionally vague — the same message is returned whether the email exists or not. This prevents attackers from using this endpoint to enumerate registered emails. A reset token is generated, stored in `users.reset_token` with a 30-minute expiry, and emailed to the owner via Nodemailer (Mailtrap Sandbox in the current dev/deployment environment).

POST /api/auth/reset-password

Auth required	None — token in body acts as proof
Actor	Business Owner
Use case	UC15 — Reset Password

Request Body

```
{
  "token": "a3f8c2e1b4d9f0a7c3e5...", // from URL query param
  "newPassword": "newSecret123"
}
```

200 OK:

```
{
  "message": "Password reset successful. You can now log in."
}
```

400 — token invalid or expired:

```
{
  "message": "Invalid or expired reset link."
}
```

Note: The token comes from the URL when the owner clicks the reset link in their email. The frontend reads it from the query string automatically and includes it in this request body. The owner never types the token manually. After a successful reset the token is cleared from the database.

Group 2 — Business Profile

GET /api/owner/business

Auth required	JWT — role = owner
Actor	Business Owner
Use case	UC8 — Owner Views Dashboard

Response — 200 OK

```
{
  "id": "uuid-here",
  "name": "Joes Plumbing",
  "slug": "joes-plumbing",
  "description": "Sydney plumber",
  "phone": "0412345678",
  "contactEmail": "joe@joesplumbing.com.au",
  "websiteUrl": "https://joesplumbing.com.au",
  "isActive": true,
  "approvalStatus": "approved",
  "rejectionReason": null,
  "createdAt": "2026-06-22T10:00:00Z"
}
```

Note: Business ID is always read from the JWT. Owners can never access another business's profile by manipulating the URL.

PATCH /api/owner/business

Auth required	JWT — role = owner
Actor	Business Owner
Use case	UC8 — Owner Updates Profile

Request Body — all fields optional

```
{
  "description": "Updated description",
  "phone": "0412345678",
  "contactEmail": "new@joesplumbing.com.au",
  "websiteUrl": "https://joesplumbing.com.au"
}
```

Response — 200 OK

```
{
  "message": "Business profile updated successfully."
}
```

Note: Business name and slug cannot be updated after registration. They are set once at registration and are permanent in MVP.

GET /api/public/businesses/:businessId

Auth required	None — public route
Actor	Customer

Use case	UC1 — Customer Loads Booking Page
----------	-----------------------------------

Response — 200 OK

```
{
  "id": "uuid-here",
  "name": "Joes Plumbing",
  "description": "Sydney plumber",
  "phone": "0412345678"
}
```

404 — business not found, not approved, or suspended:

```
{
  "message": "Business not found."
}
```

Note: The backend checks `approval_status = approved AND is_active = true` before returning data. If either check fails the response is always 404 — the customer is never told why. UC1's alternate flows A1–A4 describe three distinguishable messages for a nonexistent, pending, rejected, or suspended business; the shipped behaviour deliberately collapses all four into this one identical 404 so a customer (or attacker) cannot learn a business exists at all.

Group 3 — Availability

GET /api/owner/availability

Auth required	JWT — role = owner
Actor	Business Owner
Use case	UC6 — Owner Sets Availability

Response — 200 OK

```
{
  "availability": [
    {
      "dayOfWeek":      0,          // 0 = Monday, 6 = Sunday
      "isAvailable":    true,
      "startTime":      "09:00",
      "endTime":        "17:00",
      "breakStart":     "12:00",
      "breakEnd":       "13:00",
      "slotDurationMinutes": 30
    },
    {
      "dayOfWeek":      1,
      "isAvailable":    false,
      "startTime":      null,
      "endTime":        null,
      "breakStart":     null,
      "breakEnd":       null,
      "slotDurationMinutes": null
    }
  ]
  // ... all 7 days always returned
}
```

PUT /api/owner/availability

Auth required	JWT — role = owner
Actor	Business Owner
Use case	UC6 — Owner Sets Availability
Method note	PUT replaces the entire weekly schedule — all 7 days are always sent

Request Body — a bare array, not wrapped in { "availability": [...] } like the GET response

```
[
  {
    "dayOfWeek":      0,
    "isAvailable":    true,
    "startTime":      "09:00",
    "endTime":        "17:00",
    "breakStart":     "12:00",
    "breakEnd":       "13:00",
    "slotDurationMinutes": 30
  }
  // ... all 7 days
]
```

Response — 200 OK

```
{  
  "message": "Availability updated successfully."  
}
```

Validation Rules

- Exactly 7 days submitted (0 through 6) — no missing or duplicate days
- At least one day must have isAvailable = true
- For open days: startTime and endTime are required
- endTime must be after startTime
- If breakStart and breakEnd are set they must fall within startTime and endTime
- slotDurationMinutes must fit at least one whole slot in the open window

Note: The request/response shape is asymmetric: GET wraps the array in { "availability": [...] }, but PUT expects a bare array in the body. This was confirmed by reading the live route rather than assuming symmetry. A new business with no saved AvailabilityRule rows defaults every day to closed, not a pre-filled starter week.

Group 4 — Booking Forms & Fields

GET /api/owner/form

Auth required	JWT — role = owner
Actor	Business Owner
Use case	UC7 — Owner Builds Booking Form

Response — 200 OK

```
{
  "id": 1,
  "title": "Book a Consultation",
  "description": "Fill in the details below",
  "bookingWindowDays": 30,
  "isActive": true,
  "fields": [
    {
      "id": 1,
      "label": "Full Name",
      "fieldType": "text",
      "isRequired": true,
      "displayOrder": 1,
      "options": null,
      "isProtected": true // cannot be deleted or edited
    },
    {
      "id": 3,
      "label": "Choose your suburb",
      "fieldType": "dropdown",
      "isRequired": false,
      "displayOrder": 3,
      "options": ["Sydney CBD", "Parramatta", "Chatswood"],
      "isProtected": false
    }
  ]
}
```

PUT /api/owner/form

Auth required	JWT — role = owner
Actor	Business Owner
Use case	UC7 — Update form meta
Note	Updates form-level details only, not the fields

Request Body

```
{
  "title": "Book a Consultation",
  "description": "Fill in your details below",
  "bookingWindowDays": 14
}
```

Response — 200 OK

```
{
```

```
{
  "message": "Form updated successfully."
}
```

POST /api/owner/form/fields

Auth required	JWT — role = owner
Actor	Business Owner
Use case	UC7 — Add a field

Request Body

```
{
  "label":      "Choose your suburb",
  "fieldType":  "dropdown",          // text | textarea | dropdown | checkbox |
radio
  "isRequired": false,
  "options":    ["Sydney CBD", "Parramatta", "Chatswood"] // required for
dropdown/checkbox/radio
}
```

Response — 201 Created

```
{
  "id":          4,
  "label":       "Choose your suburb",
  "fieldType":   "dropdown",
  "isRequired":  false,
  "displayOrder": 4,
  "options":     ["Sydney CBD", "Parramatta", "Chatswood"]
}
```

PATCH /api/owner/form/fields/:id

Auth required	JWT — role = owner
Actor	Business Owner
Use case	UC7 — Edit a field

Request Body

```
{
  "label":      "Pick your area",
  "isRequired": true,
  "options":     ["Sydney CBD", "Parramatta", "Chatswood", "Bondi"]
}
```

Response — 200 OK

```
{
  "message": "Field updated successfully."
}
```

Note: Protected fields (Name, Email) cannot be edited or deleted — the backend returns 403 if attempted. `fieldType` is immutable after creation: including it in this request body returns 400, which is also why the form-builder UI disables the Type selector once a field is being edited rather than created fresh.

DELETE /api/owner/form/fields/:id

Auth required	JWT — role = owner
Actor	Business Owner
Use case	UC7 — Delete a field

Response — 200 OK

```
{
  "message": "Field deleted successfully."
}
```

PUT /api/owner/form/fields/reorder

Auth required	JWT — role = owner
Actor	Business Owner
Use case	UC7 — Reorder fields after drag and drop

Request Body

```
{
  "fields": [
    { "id": 1, "displayOrder": 1 },
    { "id": 3, "displayOrder": 2 },
    { "id": 2, "displayOrder": 3 },
    { "id": 4, "displayOrder": 4 }
  ]
}
```

Response — 200 OK

```
{
  "message": "Fields reordered successfully."
}
```

Note: The frontend sends the complete reordered list after drag and drop. Backend updates `display_order` on all affected rows in a single transaction.

Group 5 — Slots

GET /api/public/slots

Auth required	None — public route
Actor	Customer
Use case	UC2 — Customer Selects a Date and Views Slots
Query params	businessId (UUID), date (YYYY-MM-DD)
Full example	/api/public/slots?businessId=uuid&date=2026-07-15

Response — 200 OK

```
{
  "date": "2026-07-15",
  "slotDurationMinutes": 30,
  "slots": [
    { "time": "09:00", "available": true },
    { "time": "09:30", "available": true },
    { "time": "10:00", "available": false }, // already booked
    { "time": "10:30", "available": true },
    { "time": "11:00", "available": true }
  ]
}
```

Note: Slots are never stored in the database. On every request the backend fetches `availability_rules` for the day of week, generates all possible slots, removes break-time and already-booked slots, removes past slots if the date is today, and returns the full list with an `available` flag on each slot rather than hiding unavailable ones — the frontend greys them out so the customer sees the full shape of the day.

Group 6 — Bookings

POST /api/public/bookings

Auth required	None — public route
Actor	Customer
Use case	UC3 — Booking Submission and Pending Review

Request Body

```
{
  "businessId": "uuid-here",
  "formId": 1,
  "bookingDate": "2026-07-15",
  "bookingTime": "10:00",
  "customerName": "John Smith",
  "customerEmail": "john@example.com",
  "customerPhone": "0412345678", // optional
  "fieldValues": [
    { "formFieldId": 1, "value": "John Smith" },
    { "formFieldId": 2, "value": "john@example.com" },
    { "formFieldId": 3, "value": "Parramatta" }
  ]
}
```

Response — 201 Created

```
{
  "id": 456,
  "status": "pending",
  "bookingDate": "2026-07-15",
  "bookingTime": "10:00",
  "message": "Booking request sent. Joes Plumbing will confirm shortly."
}
```

409 — slot no longer available (race condition):

```
{
  "message": "This slot was just taken. Please select another time."
}
```

Note: The backend verifies the business is approved and active, re-checks slot availability server-side (never trusts the frontend), validates required fields and email format, validates field values against predefined options, and verifies the date is within the booking window — all before creating the booking and its field values in a single transaction and sending confirmation/notification emails via Nodemailer.

Concurrency note: The 409 above is backed by a Postgres partial unique index on bookings ("businessId", "bookingDate", "bookingTime") WHERE status IN ('pending','approved'), hand-written as raw SQL since Prisma's schema language has no WHERE-clause syntax for @@unique. This closes the real race a client-side pre-check alone cannot — two near-simultaneous requests can both pass an optimistic check before either writes. Both the optimistic pre-check and the DB-constraint race path return the same message deliberately, so a genuine race and an already-taken slot are indistinguishable to the client.

GET /api/owner/bookings

Auth required	JWT — role = owner
Actor	Business Owner

Use case	UC8 — Owner Views Dashboard and Manages Bookings
Query params	status (pending approved rejected cancelled), search, page
Pagination	25 per page

Response — 200 OK

```
{
  "bookings": [
    {
      "id": 456,
      "customerName": "John Smith",
      "customerEmail": "john@example.com",
      "bookingDate": "2026-07-15",
      "bookingTime": "10:00",
      "status": "pending",
      "createdAt": "2026-07-10T08:00:00Z"
    }
  ],
  "total": 1,
  "page": 1,
  "totalPages": 1,
  "counts": {
    "pending": 1,
    "approved": 0,
    "rejected": 0,
    "cancelled": 0,
    "all": 1
  }
}
```

Note: Backend always filters by the JWT business ID. Owners can never see another business's bookings regardless of query parameters. The counts object is status-blind — computed via a separate countsWhere query (the codebase's first use of Prisma's groupBy) — so the five filter pills on the owner bookings screen show real numbers on every pill even while filtered to just one status, instead of zeroing out the other four.

GET /api/owner/bookings/:id

Auth required	JWT — role = owner
Actor	Business Owner
Use case	UC5 — Owner Reviews a Pending Booking

Response — 200 OK

```
{
  "id": 456,
  "customerName": "John Smith",
  "customerEmail": "john@example.com",
  "customerPhone": "0412345678",
  "bookingDate": "2026-07-15",
  "bookingTime": "10:00",
  "status": "pending",
  "ownerNotes": null,
  "createdAt": "2026-07-10T08:00:00Z",
  "fieldValues": [
    { "label": "Choose your suburb", "value": "Parramatta" }
  ]
}
```

PATCH /api/owner/bookings/:id/approve

Auth required	JWT — role = owner
Actor	Business Owner
Use case	UC5 — Owner Approves a Booking

Request Body

```
{
  "ownerNotes": "See you then."    // optional
}
```

Response — 200 OK

```
{
  "message": "Booking approved. Customer has been notified."
}
```

PATCH /api/owner/bookings/:id/reject

Auth required	JWT — role = owner
Actor	Business Owner
Use case	UC5 — Owner Rejects a Booking

Request Body

```
{
  "ownerNotes": "Sorry we are fully booked that week."    // optional
}
```

Response — 200 OK

```
{
  "message": "Booking rejected. Customer has been notified."
}
```

Note: Rejecting a booking releases the slot. It becomes available again for new customer bookings.

PATCH /api/owner/bookings/:id/cancel

Auth required	JWT — role = owner
Actor	Business Owner
Use case	UC8 — Owner Cancels an Approved Booking

Request Body

```
{
  "ownerNotes": "Apologies, unforeseen circumstances."    // optional
}
```

Response — 200 OK

```
{
```

```
"message": "Booking cancelled. Customer has been notified."  
}
```

Note: *Cancelling an approved booking releases the slot and sends a cancellation email to the customer via Nodemailer.*

Group 7 — Admin

GET /api/admin/me

Auth required	JWT — role = admin
Actor	Platform Administrator
Use case	Supports the admin dashboard identity block (no dedicated UC — added post-launch)

Response — 200 OK

```
{
  "name": "Platform Admin",
  "email": "admin@orbisscheduler.com"
}
```

Note: The 30th and newest route. None of the original 7 admin routes returned the logged-in admin's own identity, so the admin sidebar had nothing to render once it needed a name/email block the same way the owner sidebar already shows owner.email. This route mirrors GET /api/owner/business in purpose, scoped to the admin's own user row instead of a business.

GET /api/admin/stats

Auth required	JWT — role = admin
Actor	Platform Administrator
Use case	UC12 — Admin Views Platform Overview

Response — 200 OK

```
{
  "totalBusinesses": 42,
  "pendingRegistrations": 3,
  "approvedBusinesses": 35,
  "rejectedBusinesses": 4,
  "suspendedBusinesses": 2,
  "totalBookingsLifetime": 1280,
  "bookingsThisWeek": 47
}
```

Note: "This week" is a rolling 7-day window, not the calendar week — deliberately not reusing the AvailabilityRule.dayOfWeek Monday = 0 convention, since a trailing activity metric shouldn't reset to near-zero every Monday morning. Only two fixed windows exist today (all-time, rolling 7-day); a selectable custom date range is a deferred future improvement.

GET /api/admin/businesses

Auth required	JWT — role = admin
Actor	Platform Administrator
Use case	UC13 — Admin Views All Businesses
Query params	status, isActive, search, page
Pagination	25 per page

Response — 200 OK

```
{
```

```

"businesses": [
  {
    "id": "uuid-here",
    "name": "Joes Plumbing",
    "ownerName": "Joe Smith",
    "ownerEmail": "joe@example.com",
    "approvalStatus": "pending",
    "isActive": true,
    "totalBookings": 0,
    "createdAt": "2026-07-01T08:00:00Z"
  }
],
"total": 42,
"page": 1,
"totalPages": 2
}

```

GET /api/admin/businesses/:id

Auth required	JWT — role = admin
Actor	Platform Administrator
Use case	UC11 — Admin Reviews Registration Details

Response — 200 OK

```

{
  "id": "uuid-here",
  "name": "Joes Plumbing",
  "slug": "joes-plumbing",
  "description": "Sydney plumber",
  "phone": "0412345678",
  "contactEmail": "joe@joesplumbing.com.au",
  "websiteUrl": "https://joesplumbing.com.au",
  "approvalStatus": "pending",
  "rejectionReason": null,
  "isActive": true,
  "createdAt": "2026-07-01T08:00:00Z",
  "owner": {
    "name": "Joe Smith",
    "email": "joe@example.com"
  },
  "totalBookings": 0
}

```

PATCH /api/admin/businesses/:id/approve

Auth required	JWT — role = admin
Actor	Platform Administrator
Use case	UC11 — Admin Approves Registration

Request Body

No body required. Admin confirms via the action URL.

Response — 200 OK

```
{
  "message": "Business approved. Owner has been notified."
}
```

Note: Sets `approval_status` to `approved`. Sends approval email to the owner via Nodemailer. The owner can now log in and access the dashboard.

PATCH `/api/admin/businesses/:id/reject`

Auth required	JWT — role = admin
Actor	Platform Administrator
Use case	UC11 — Admin Rejects Registration

Request Body

```
{
  "rejectionReason": "Business type not supported on this platform." // required
}
```

Response — 200 OK

```
{
  "message": "Business rejected. Owner has been notified."
}
```

Note: Rejection reason is required. Sets `approval_status` to `rejected` and stores the reason. Sends rejection email to owner. The owner cannot log in or reapply with the same email unless an admin clears the rejection.

PATCH `/api/admin/businesses/:id/suspend`

Auth required	JWT — role = admin
Actor	Platform Administrator
Use case	UC14 — Admin Suspends a Business

Request Body

No body required.

Response — 200 OK

```
{
  "message": "Business suspended successfully."
}
```

Note: Sets `is_active` to `false`. Owner is blocked from logging in. Public booking page returns 404. Existing approved bookings are preserved — suspending a business does not cancel customer appointments. No suspension-notification email is sent to the owner; this is silent by design per the use case doc, flagged as a possible future addition.

PATCH `/api/admin/businesses/:id/activate`

Auth required	JWT — role = admin
Actor	Platform Administrator
Use case	UC14 — Admin Reactivates a Business

Request Body

No body required.

Response — 200 OK

```
{  
  "message": "Business activated successfully."  
}
```

Note: Sets *is_active* to true. Owner regains full access immediately. Public booking page becomes functional again.

3. HTTP Status Code Reference

Code	Meaning in Orbis Scheduler
200	Request succeeded — data returned or action completed
201	Resource created successfully — booking submitted, field added
400	Bad request — validation failed, token expired, invalid input
401	Unauthenticated — no token or invalid token provided
403	Forbidden — wrong role, account pending/suspended/rejected, or a protected-field edit/delete attempt
404	Not found — resource does not exist, or a cross-tenant / not-yet-approved business is not accessible
409	Conflict — slot already taken by another booking
500	Server error — logged internally, generic message returned to client

4. Documentation Corrections

These are not implementation gaps — they are places where an earlier use case document or an earlier draft of this API reference no longer describes the shipped, intentional behaviour. Consolidated from docs/FUTURE_IMPROVEMENTS.md and docs/IMPLEMENTATION_NOTES.md, since they came up repeatedly while building the frontend against those same documents.

Source doc	What it says	What actually ships
UC9, alt. flow A3	Cross-tenant booking access returns 403 Forbidden	404, everywhere, deliberately — a 403 would confirm the resource exists on another tenant; 404 does not
UC1, alt. flows A1–A4	Three distinguishable messages for a nonexistent / pending / rejected / suspended business	One identical 404 “Business not found.” for all four cases
UC2 A3 / UC3 A1	Two different quoted strings for the same slot-race event	One string: “This slot was just taken. Please select another time.” on both the optimistic pre-check and the DB-constraint race path
UC2 A2	“Invalid value for [field label]”	“\${field.label} must be one of the provided options”
UC10	Section B is an iframe HTML snippet	A <script> tag opening a modal overlay — a plain iframe gets squeezed by whatever layout surrounds it
Earlier drafts of this document	Header said 27 routes; the endpoint table summed to 29	Real total is 30 — GET /api/admin/me was added after both were written

Provenance: None of the corrections above were made to the binary use-case .docx source in this repo — no editable source exists for it here. CLAUDE.md and docs/FUTURE_IMPROVEMENTS.md carry the corrections instead, and this revision of the API reference now reflects them directly.

5. Key Design Decisions

Decision	Reasoning
Route prefixes by actor	Four distinct prefixes (/auth, /public, /owner, /admin) make auth enforcement simple — middleware applies at the prefix level, not per-route
JWT from token, never URL	Business identity always comes from the JWT payload. URL parameters are ignored on protected routes. Prevents IDOR attacks.
PUT for availability	The full week is always submitted as one payload. PUT (replace entirely) is semantically correct — PATCH (partial update) would complicate partial-day logic.
No slot storage	Slots are calculated on demand from availability_rules. Pre-storing them creates stale data problems. On-demand calculation is always accurate.
409 for slot conflicts	A race condition where two customers pick the same slot simultaneously is a conflict, not a bad request. Backed by a Postgres partial unique index, not just an optimistic pre-check.
Vague forgot-password response	The same response is returned whether the email exists or not. Prevents attackers from using the endpoint to enumerate registered emails.
isProtected on form fields	A real boolean column (not string-matching on label) flags Name/Email so the backend blocks delete/edit attempts on these fields regardless of how the request is made.
Paginated booking lists	Bookings are paginated at 25 per page from day one. Loading all bookings in one query is a performance problem as data grows.
counts on GET /owner/bookings	Computed via a status-blind groupBy query so the dashboard's filter pills show real numbers on every status, not just the one currently filtered to.
GET /admin/me added post-launch	The admin sidebar needed an identity block mirroring the owner sidebar's owner.email, and none of the original 7 admin routes exposed the logged-in admin's own name/email.

End of document

Hussain Jameel · ITA602 · Torrens University Australia